

Almost Self-Improving Agentic Workflows in Flutter

Łukasz Wiśniewski



Agenda





Łukasz Wiśniewski

INDIE DEVELOPER #OpenToWork

a.k.a Unemployed

18+ Years

Mobile Development

7 Years

Flutter Experience

X / GITHUB vishna

LINKEDIN lukaszwisniewski

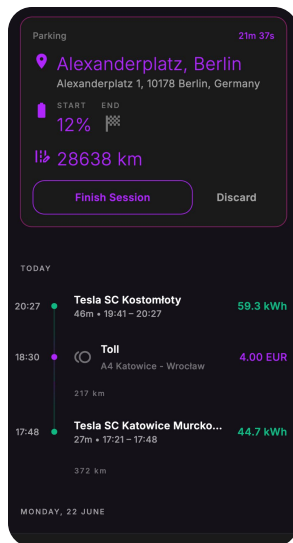
WEBSITE vishna.dev



AmperRank

Flutter

MOBILE

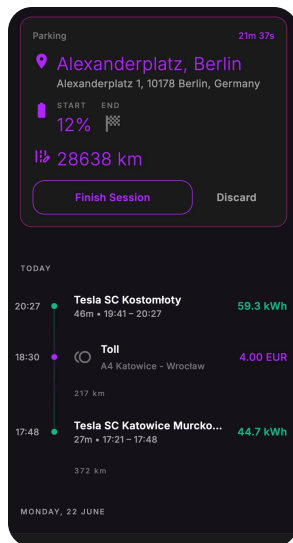


- Track charging sessions.
- Compare operators
- Save money.

AmperRank

Flutter

MOBILE



Serverpod

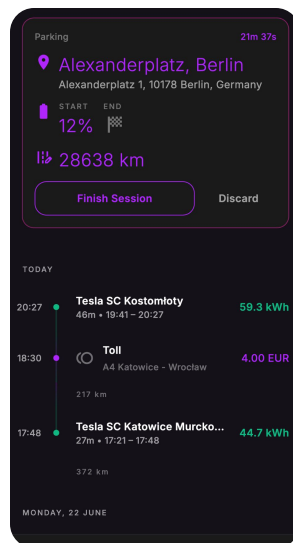
BACKEND



AmperRank

Flutter

MOBILE



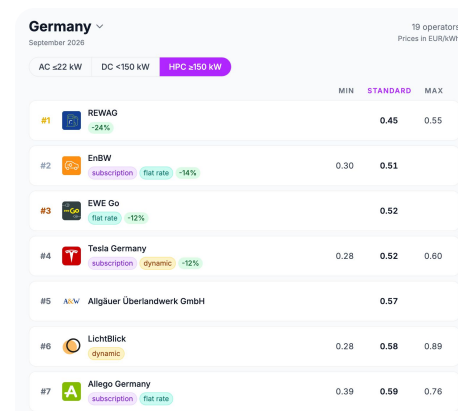
Serverpod

BACKEND



Jaspr

WEB



”

...proven track record of
implementing self-improving
agentic flows

Workflow

Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager
2. Implement required code changes
3. Commit changes with ticket ref

Example Prompt

```
> /take-task PROJ-1042
```

Agentic Workflow v1

INPUTS

Task

Target goal or prompt

Source Code

Existing codebase context

Skills & Rules

Guidelines & tools

PROCESSING LOOP



AGENT

Execution & Reasoning

OUTPUTS

Task Completed

Updated Source Code



Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager
2. Implement required code changes
3. Commit changes with ticket ref

Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager
2. Implement required code changes
- 3. Loop until the code compiles**
4. Commit changes with ticket ref

Agentic Workflow v2



Feedback Loops

Automated & Fast Loops



Compiler



Lint / Custom Lint



Automated Tests

Unit

Code coverage

Goldens

E2E

Custom scripts

Human & Outer Loops



PR Review

Peer review, architectural feedback & standards



App Feedback

Internal testing, dogfooding & UX evaluation



App Store Review

End-user ratings, store feedback & telemetry

Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager.
2. Implement required code changes.
3. Loop until:
 - code compiles
 - linter pass
 - test pass
 - PR approved
4. Merge PR.

Agentic Workflow v3



Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager.
2. Implement required code changes.
3. Loop until:
 - code compiles
 - linter pass
 - test pass
 - PR approved
4. Merge PR.
5. Run `/retro [session-ID]`

Automated Retrospective

01

Inputs & Analysis

Gather context across sources



PR Details & Code Diff

Analyzes pull request changes and code deltas



Reviewer Feedback

Processes comments and discussion threads



Execution Logs

Inspects runtime outputs and test results



02

Lesson Extraction & Updates

Refine agent rules/skills and project docs

Synthesize Findings

- **Wins:** Efficient patterns & tools
- **Mistakes:** Misunderstandings/errors
- **Patterns:** Recurring project rules

Update Knowledge & Skills

Rule Base (.agent/rules/):

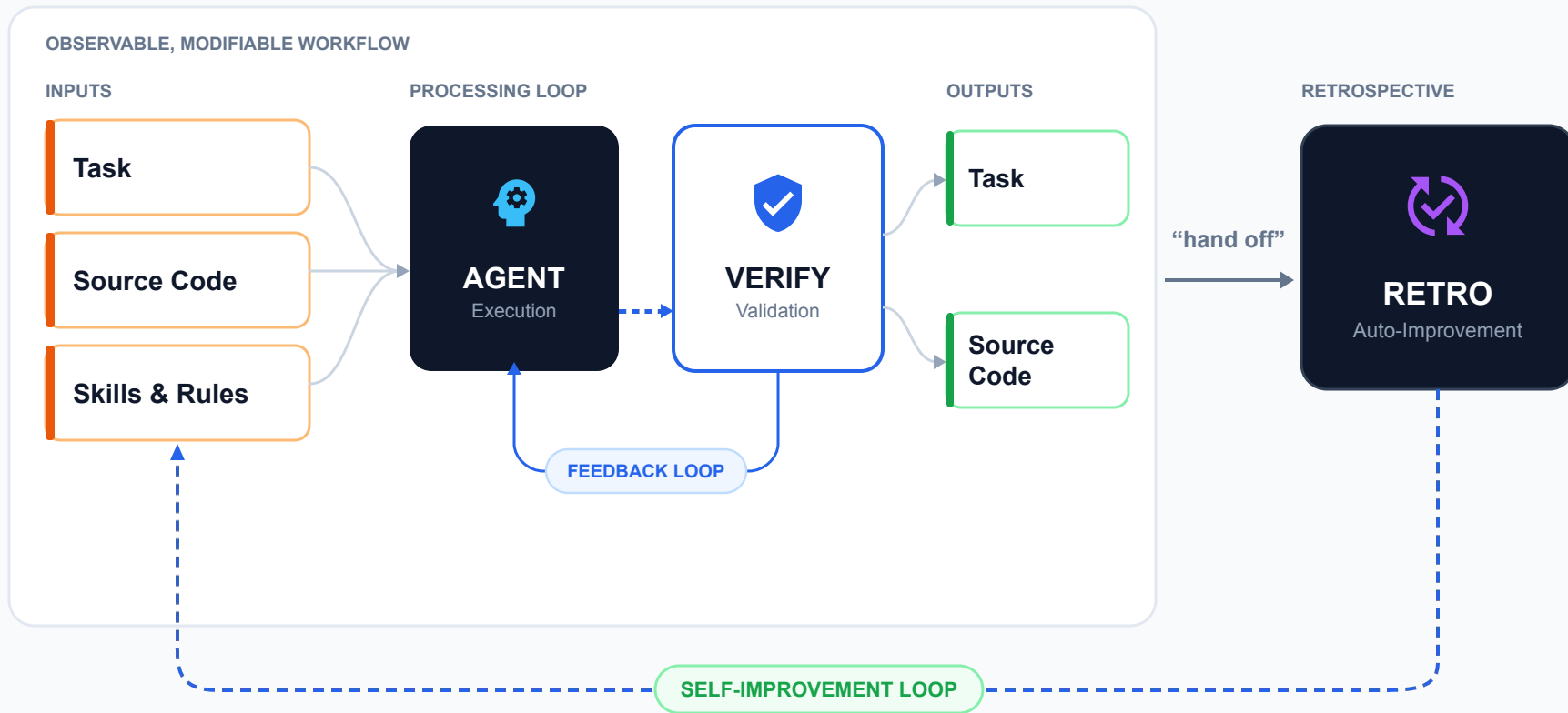
Do **Don't** Remember

Update Skills (.agent/skills/)

Documentation Alignment

Synchronize repository documentation with new standards

Agentic Workflow v4



Feedback to Rules

Code

```
Text("Hello World");
```

Feedback to Rules

Code

```
Text("Hello World");
```



Human Review

*"Hardcoded text, no styling, pls
fix"*

Feedback to Rules

Code

```
Text("Hello World");
```

Code After Feedback

```
Text(  
    strings.helloWorld,  
    style: theme.h1  
);
```

Feedback to Rules

Code

```
Text("Hello World");
```

Code After Feedback

```
Text(  
  strings.helloWorld,  
  style: theme.h1  
);
```



RULES.md

Updated Knowledge Base

.agent/rules/

DO

Use localized string references
(`strings.*`)

DON'T

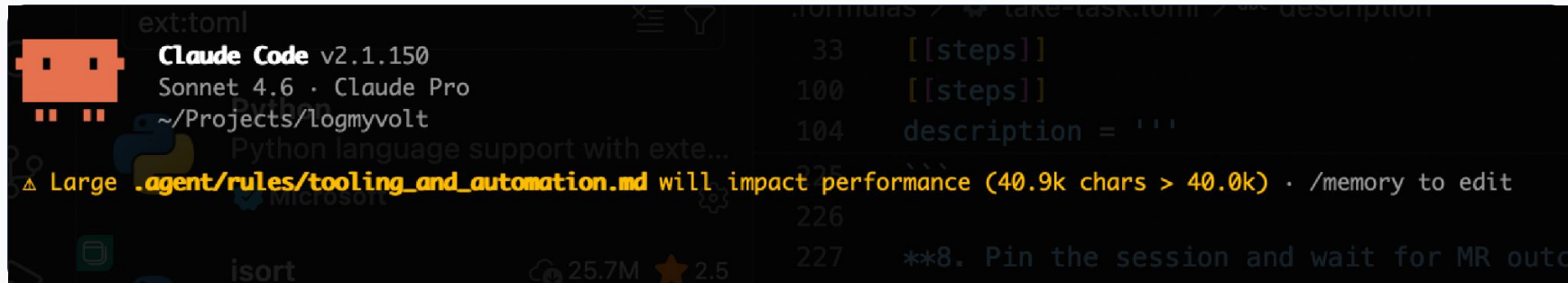
Hardcode raw string literals directly
inside UI widgets

REMEMBER

Apply centralized theme typography
styles (`theme.h1`)

Performance Impact of Large Rule Files

Exceeding context memory limits degrades agent performance and execution speed



rules.md

Project memory

REMEMBER

Rules are there for a reason, ignoring them will upset the human.



Probabilistic vs. Deterministic Enforcement

Transitioning prompt instructions into hard static analysis rules

PROBABILISTIC

rules.md

Context guidelines for AI agent behavior

● ● ● rules.md

DON'T: Hard code plain text strings with components like `Text`

DO: Use `context.strings` to get localization object

REMEMBER: Apply theme to `Text` using `context.theme`



DETERMINISTIC

custom_lint

Compiler-enforced rules for instant feedback

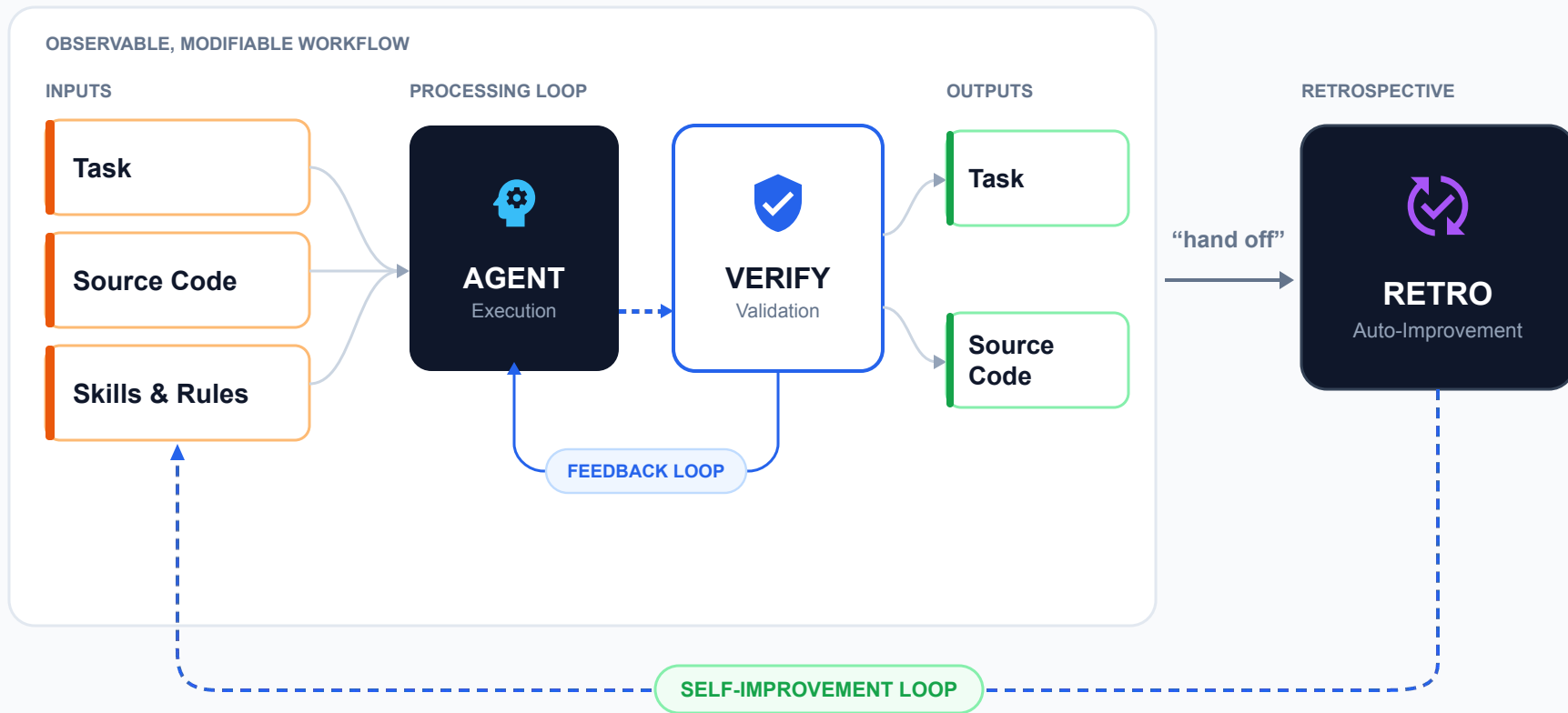
● ● ● analysis_options.yaml

✓ **AvoidHardcodedText**

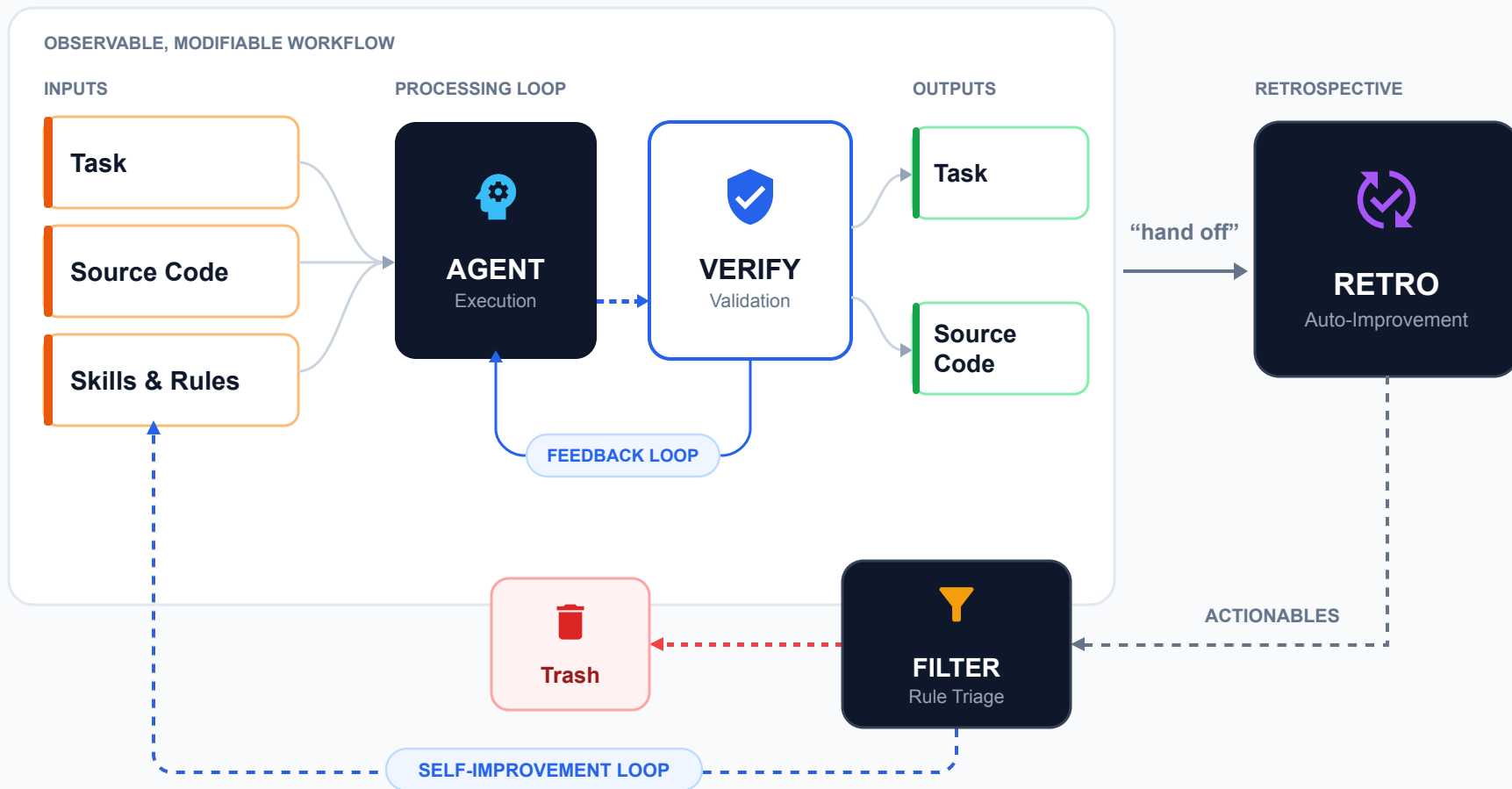
✓ **AlwaysUseI18nStrings**

✓ **AlwaysUseDesignSystemTextStyle**

Agentic Workflow v4



Agentic Workflow v5



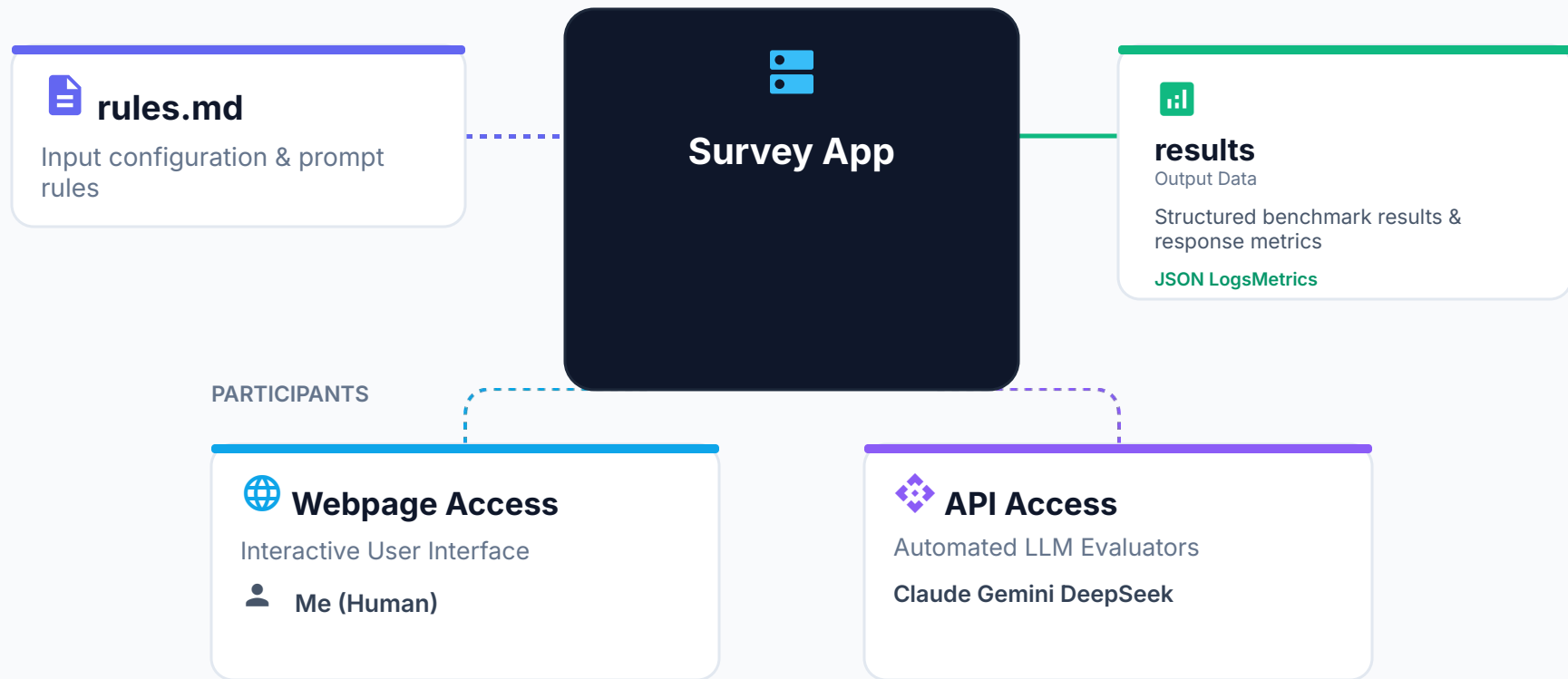
Filter

Evaluating and triaging rules into actionable execution paths



Filter Experiment - A Survey

System pipeline connecting input rules, webpage & API model access to final results



Survey Results

Consensus breakdown across 26 evaluated rules between Human & LLM participants

HUMAN UNIT

 Łukasz

CLANKER BUDDIES (LLM EVALUATORS)

 Claude • Gemini • DeepSeek



Alignment (31%)

Rules where Human Unit & LLMs reached agreement

8

Clanker Alignment (54%)

Rules where AI models agreed among themselves

14

No Consensus (15%)

Rules with split or conflicting evaluations

4



95
%
TRASH

5%

GOOD

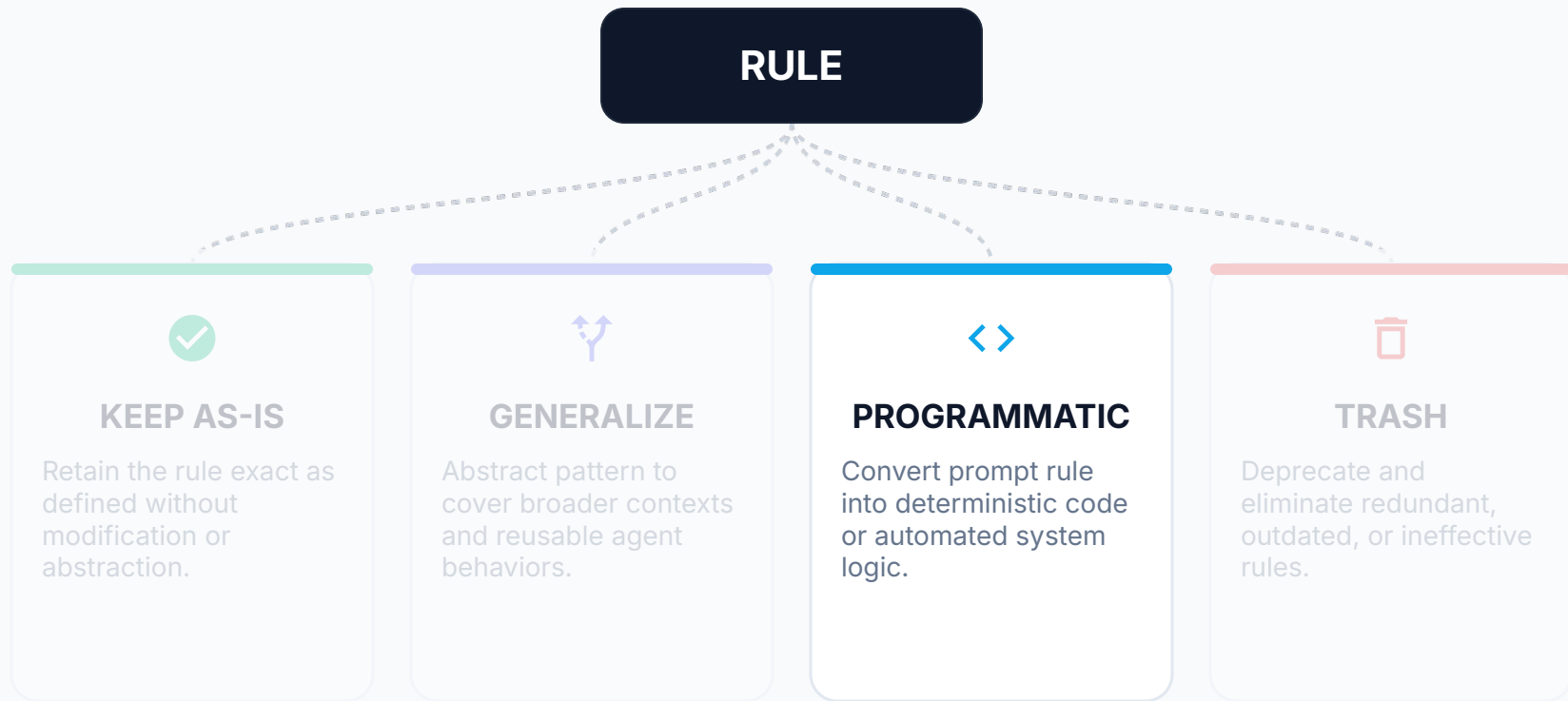
Rule Categorization Workflow

Evaluating and triaging rules into actionable execution paths



Rule Categorization Workflow

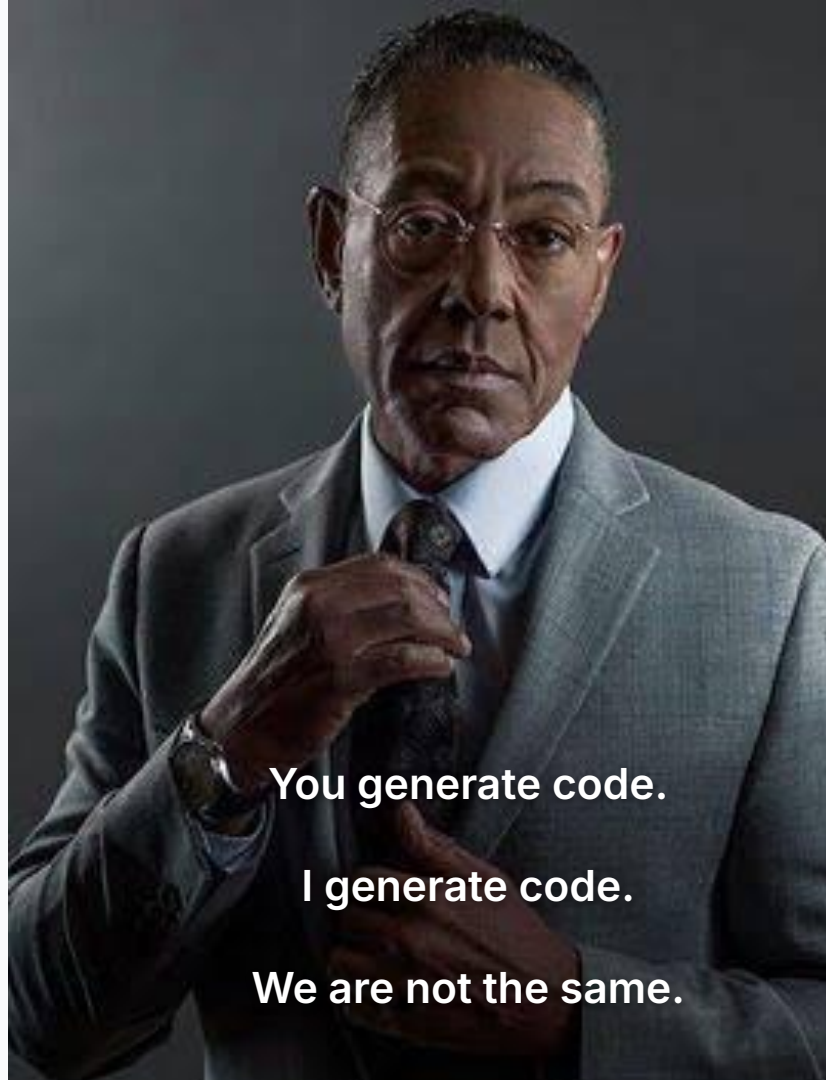
Evaluating and triaging rules into actionable execution paths



Improvements

Code generation

LLM & build_runner



You generate code.

I generate code.

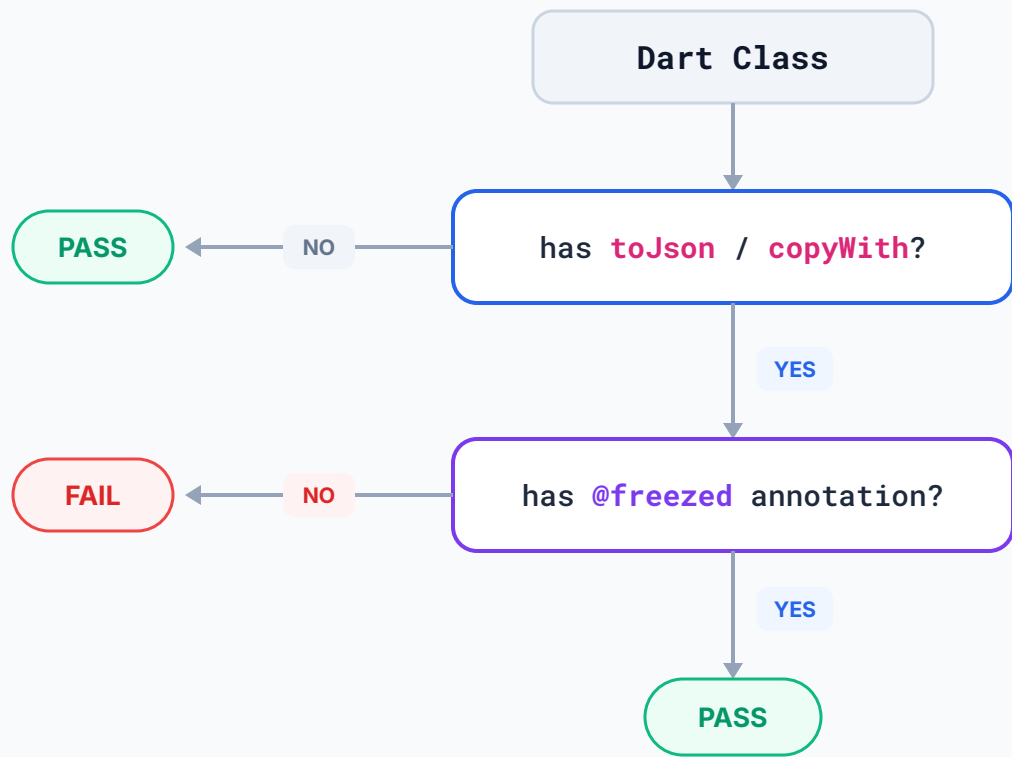
We are not the same.

rules.md

Project memory

DON'T Write data classes manually

REMEMBER Always use freezed for data classes



Navigation

go_router



rules.md

Project memory

REMEMBER

When using context.go using always check routes.dart for defined paths instead of hallucinating them.

pub.dev

Sign in Help

go_router_builder 4.5.0

Published 6 days ago • flutter.dev

SDK | FLUTTER | PLATFORM | ANDROID | IOS | LINUX | MACOS | WINDOWS

301

[Readme](#) | [Changelog](#) | [Example](#) | [Installing](#) | [Versions](#) | [Scores](#)

Usage

Dependencies

To use `go_router_builder`, you need to have the following dependencies in `pubspec.yaml`.

```
dependencies:  
  # ...along with your other dependencies  
  go_router: ^16.2.0  
  
dev_dependencies:  
  # ...along with your other dev-dependencies  
  build_runner: ^2.6.0  
  go_router_builder: ^4.0.1
```

Source code

Instructions below explain how to create and annotate types to use this builder. Along with importing the

301

LIKES

160

POINTS

267k

DOWNLOADS

Documentation

[API reference](#)

Publisher

[flutter.dev](#)

Weekly Downloads



2025.10.12 - 2026.09.06

Metadata

A builder that supports

Design System

widgetbook



rules.md

Project memory

DON'T

Directly use material or cupertino components in the main app

DO

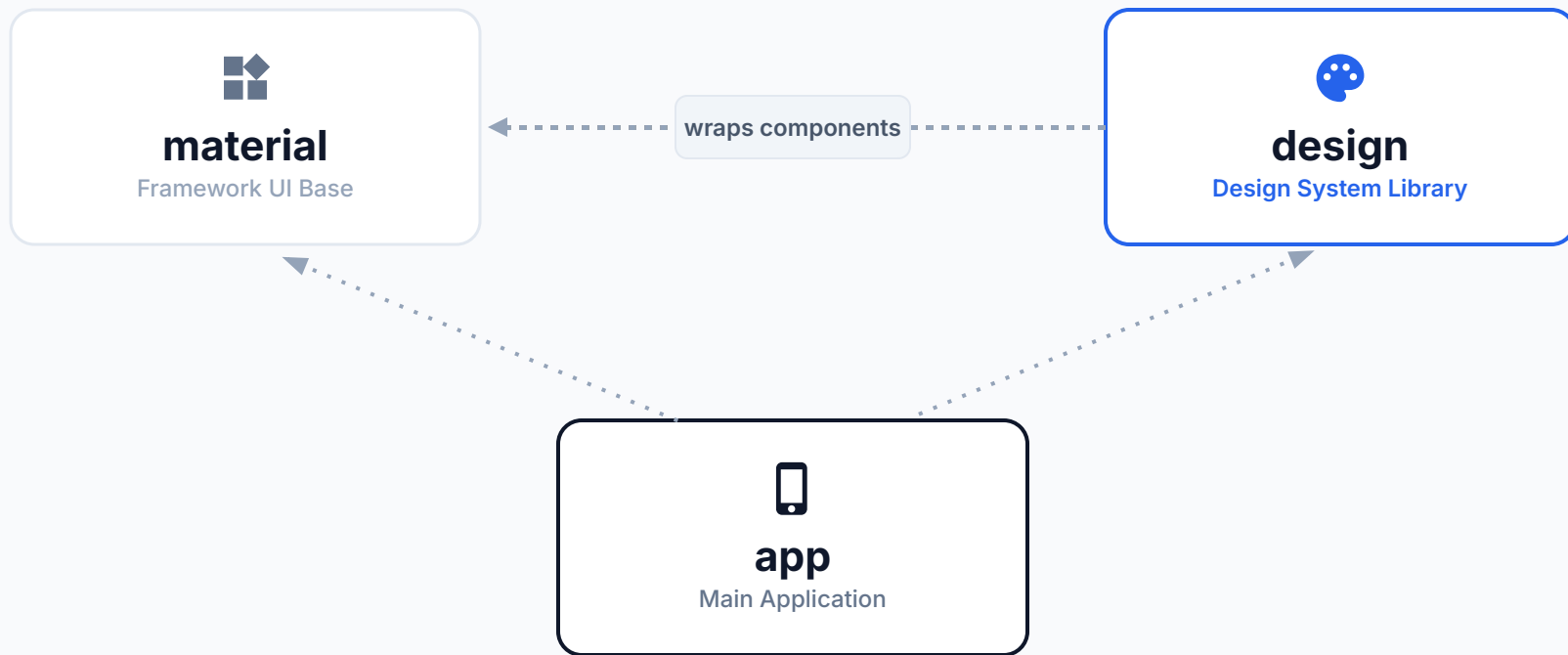
When building a new component, add it as widgetbook use case first

REMEMBER

When building a new page or widget to the main app, make sure to use components from design system

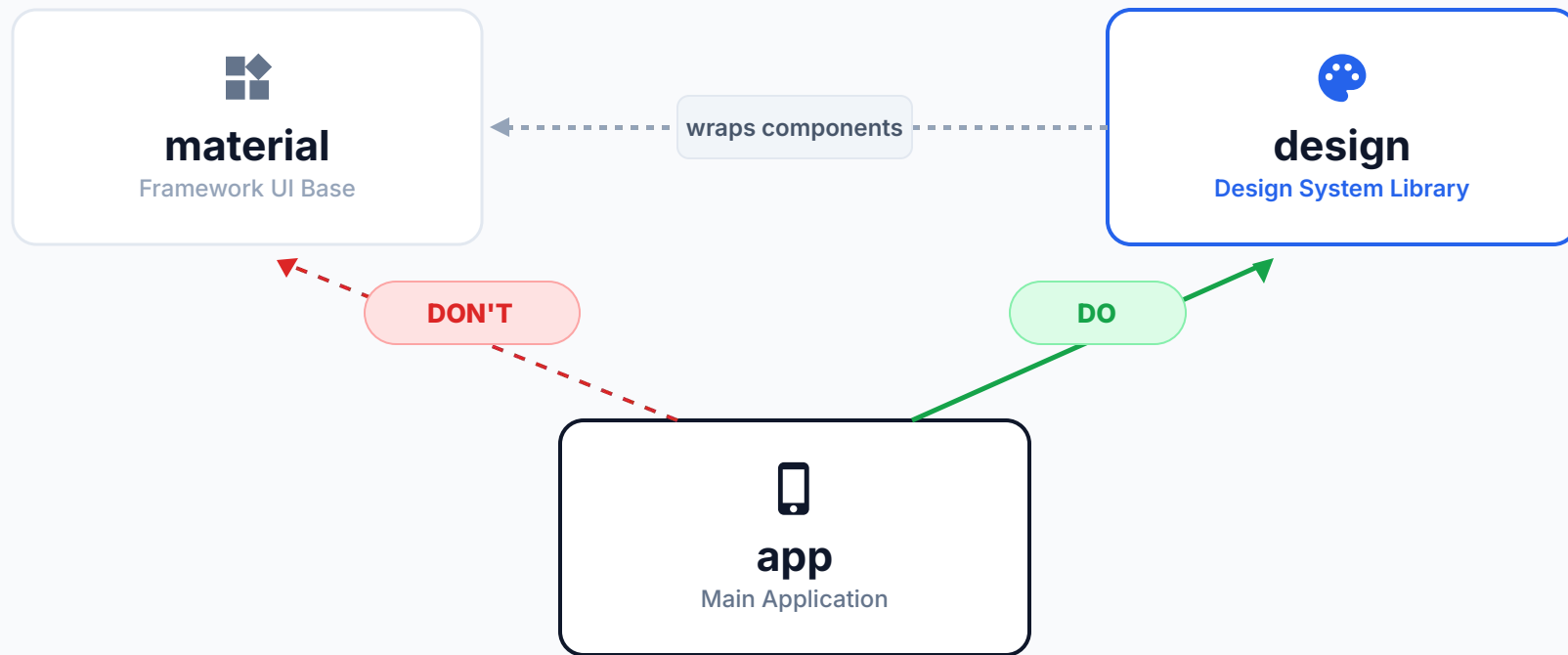
Design System

Component dependency and usage boundaries



Design System

Component dependency and usage boundaries



Organization

feature folders, naming conventions, etc.



rules.md

Project memory

DO

When creating feature directories, strictly follow the **lib/src/features/[feature]/{controllers,pages,widgets}** pattern and include **.gitkeep** files in empty subdirectories to ensure the structure is preserved in version control.

pub.dev

Search Sign in Help

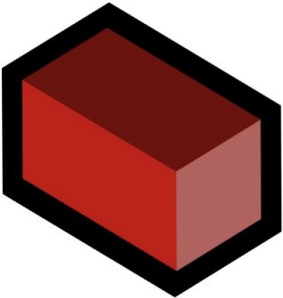
mason 0.1.2

Published 9 months ago • [brickhub.dev](#)

SDK DART FLUTTER PLATFORM ANDROID IOS LINUX MACOS WEB WINDOWS

838

[Readme](#) Changelog Example Installing Versions Scores



mason

pub v0.1.2 mason passing coverage 100% license MIT

838

LIKES

150

POINTS

424k

DOWNLOADS

Documentation

Documentation

API reference

Publisher

[brickhub.dev](#)

Weekly Downloads

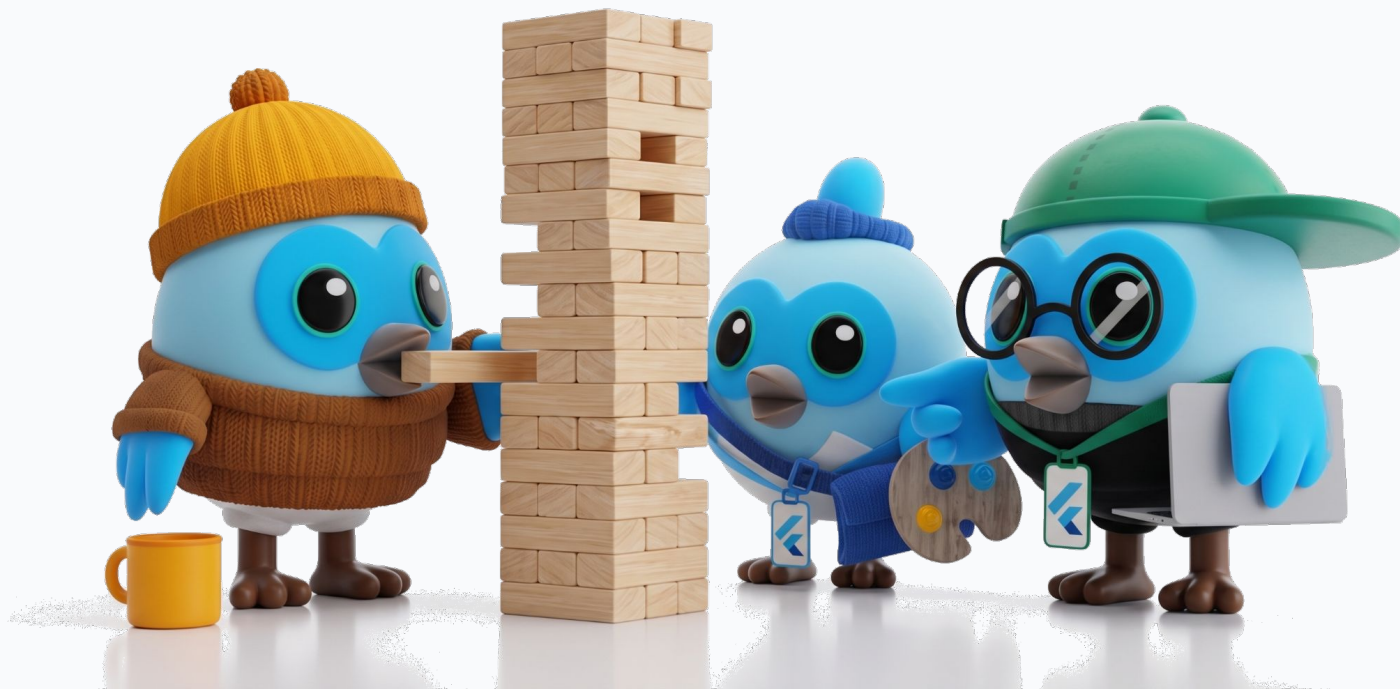


2025.10.14 – 2026.09.08

Metadata

Architecture

State Management



rules.md

Project memory

DON'T

Import material package into bloc/cubit classes.

DO

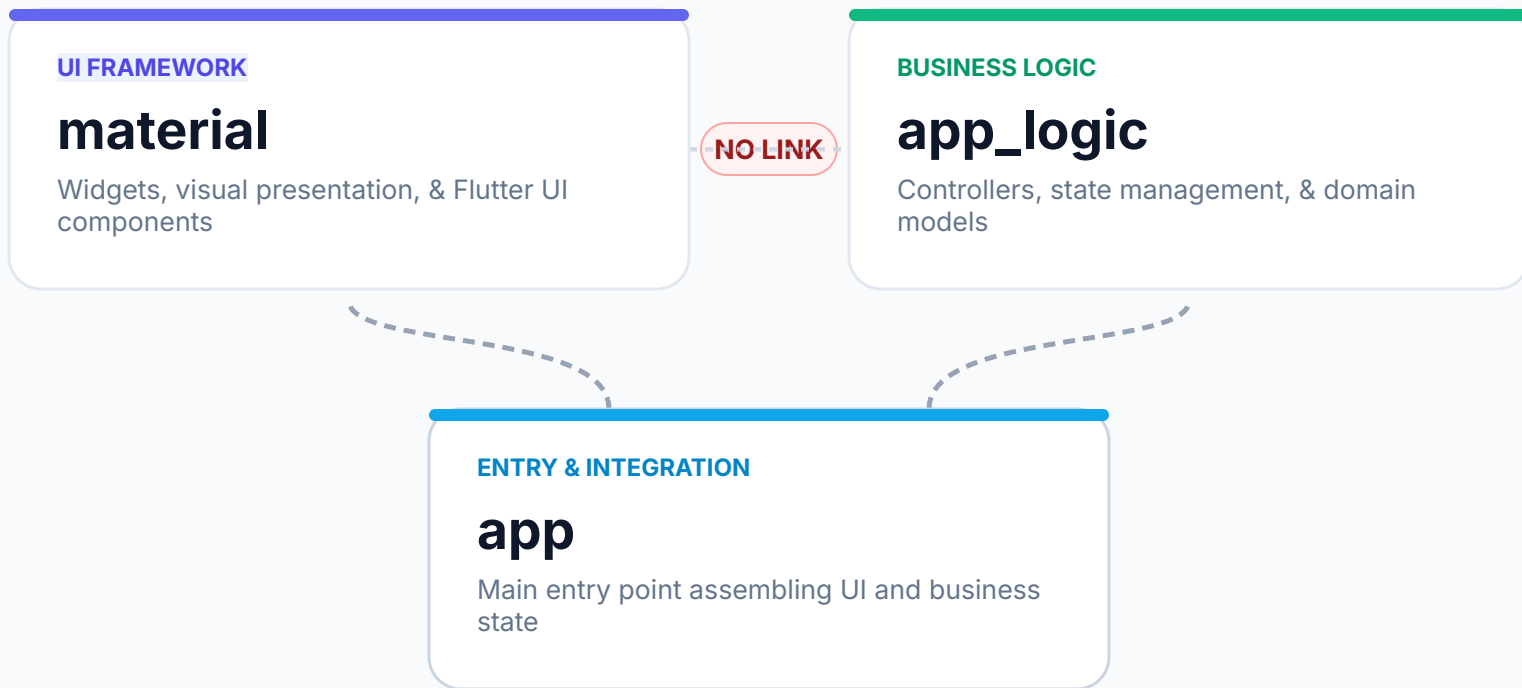
Create bloc classes only in the business_logic folder.

REMEMBER

UI code and business logic code should not mix.

Module Dependency Architecture

Strict decoupling of UI framework components and application business logic



Workflow

as markdown?



Skill: take-task [ID]

Workflow

1. Read task [ID] details & requirements from ticket manager.
2. Implement required code changes.
3. Loop until:
 - code compiles
 - linter pass
 - test pass
 - PR approved
4. Merge PR.
5. Run /retro [session-ID]

workflow.ts

```
// Skill: take-task [ID]
```

```
export async function takeTask(taskId: string) {  
  // 1. Read task details & requirements from ticket manager  
  const task = await fetchTask(taskId);  
  // 2. Delegate implementation & loop until tests pass  
  const result = await agent.execute({ task, autoFix: true });  
  // 3. Merge PR & commit changes with ticket ref  
  await gitCommit(task.ticketRef);  
  // 4. Run retrospective & improve workflow  
  await retro(task, result);  
}
```



**Unit Testing
Workflow**



**Swappable Agents
& Envs**



**Performance
Metrics**

Summary

Thank You!

